Backtracking — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving any backtracking problem. They are not tied to one question type. They help you recognize, design, and implement backtracking from scratch.

🧠 How to Recognize a Backtracking Problem

A problem is very likely backtracking if it contains phrases like:

- "Find **all** ways / combinations / subsets / arrangements"
- "Generate..."
- "Return all possible..."
- "Place / choose / arrange..."
- Small constraints (often $n \leq 15$–20)
- Brute force seems exponential but pruning is possible

These problems are asking you to explore a **decision tree**.

😨 What Backtracking Really Is

You are walking a tree of choices.

At each step you ask:

> "What choice do I make next?"

You go deeper, then you come back and undo that choice.

That is backtracking.

👎 The Universal Skeleton

```
void backtrack(state) {

    if (base_case) {
        save_answer;
        return;
    }
```

```
    for (each possible choice from current state) {

        if (invalid choice) continue;

        make the choice;

        backtrack(new state);

        undo the choice;
    }
}
```

This skeleton works for almost every backtracking problem.

🤔 How to Identify the "State"

State is what defines where you are in the tree.

Problem	State
Subsets	current index, current subset
Combination sum	index, current subset, remaining target
Permutations	current permutation, visited[]
N Queens	board configuration, row
Sudoku	board

If you can describe where you are, you can backtrack.

😌 The Golden Rule

> If you change something before recursion, undo it after.

Examples:

Change	Undo
push_back	pop_back
visited[i] = true	visited[i] = false
sum += x	sum -= x

Change	Undo
board[r][c] = 'Q'	board[r][c] = '.'

🧠 Designing the For-loop

Ask:

> From here, what are my possible next choices?

That becomes your for-loop.

Problem	For loop over
Subsets	remaining array elements
Permutations	all unused elements
N Queens	all columns in this row
Sudoku	numbers 1 to 9

😭 When to Skip a Choice

Common reasons:

1. Duplicate choice at same depth
2. Choice violates constraint
3. Choice exceeds limit (like target)

These become `continue` or `break`.

😾 Duplicate Rule (very common)

If input has duplicates and answers must be unique:

```
if (i > start && arr[i] == arr[i-1]) continue;
```

Meaning:

> Do not start two branches with the same value at the same depth.

😖 Two Styles of Backtracking

1. Pick / Not Pick (binary tree)

Used when decision per element is yes/no.

2. For-loop choice style (cleaner)

Used when choosing the next element from many.

Both represent the same tree. The for-loop style is usually preferred.

👂 Target / Sum Problems Trick

Never track sum manually.

Track **remaining target** instead.

Safer and avoids state bugs.

🧠 Mental Checklist Before Coding

1. What is my state?
2. What are my choices from here?
3. What is my base case?
4. What do I change before recursion?
5. How do I undo after?

If you can answer these, you can write the code.

🤔 Final Pattern Recognition Summary

Backtracking problems typically:

- Ask for all possibilities
- Require exploring combinations or arrangements
- Have constraints small enough for exponential exploration
- Need pruning to avoid invalid paths
- Require careful state management and undoing changes

You are exploring a maze, leaving breadcrumbs, and picking them back up.

That is backtracking.

😺 Universal For-loop Template (Combinations / Subsets Family)

```
void backtrack(int start, vector<int>& arr,
               vector<int>& curr, ...) {

    // base case if needed

    for (int i = start; i < arr.size(); i++) {

        if (i > start && arr[i] == arr[i-1]) continue;

        curr.push_back(arr[i]);

        backtrack(i + 1, arr, curr, ...);

        curr.pop_back();
    }
}
```

Use this as the default starting point for many problems.